RajyaRank

DEPLOYMENT PLAN

Staging & Production Deployment Plan

A concrete, repo-grounded plan for standing up a fresh Staging environment and promoting to Production — including two blocking gaps found while auditing the current deployment tooling that must be closed first.

Prepared for: RajyaRank engagement

Scope: apps/api, apps/worker, apps/web, apps/admin — full stack

Source: docs/environments.md, docs/ci-cd.md, docs/backup-restore.md, SECURITY.md, docs/checklists.md, docs/monitoring.md, .env.*.example, infra/docker/*, infra/deployment/migrate.sh, apps/api/prisma/seed.ts

1 · Purpose & how this plan was built

This plan is assembled entirely from what already exists in the RajyaRank repository — the environment matrix, the CI/CD workflows, the Docker/compose files, the seed script's production guard, and the security/launch checklists already written by the team — plus two concrete gaps found by directly reading that tooling rather than assuming it works. It is not a generic "how to deploy a Next.js/NestJS app" template; every step below maps to a real file in this repo.

What already exists and works: a documented four-tier environment matrix (local / staging / preproduction / production), env templates per tier, Dockerfiles for all four services, a one-shot `migrate` job pattern, a GitHub Actions workflow that builds and publishes all four images to GHCR on every merge to `main`, a registry-based compose file to pull and run those images on a target host, health endpoints (`/healthz` , `/readyz` , `/metrics`), and a written backup/restore procedure with RTO/RPO targets.

What does not exist yet (found during this audit, not previously documented):

1. **No Prisma migration history.** `apps/api/prisma/migrations/` does not exist. Every schema change to date — across all eight build phases plus this session's Course Studio work — was applied with `prisma db push` against the local dev database, never `prisma migrate dev` . The deploy tooling's `migrate.sh` calls `prisma migrate deploy` , which replays that history — with zero migrations, this step is a silent no-op and a fresh Staging/Production database would never receive the schema at all.
2. **No bootstrap path for the first Super Admin.** `seed.ts` deliberately seeds demo staff accounts only when `NODE_ENV !== 'production'` — correct behaviour so that no demo credentials ever exist in a real environment, but it means a fresh Staging/Production database ends up with reference data only (states/exams/roles) and **zero staff accounts**. Every other admin action requires being logged in as staff already, so there is currently no documented way to create the very first Super Admin on a new environment.

Both are addressed as the first two action items in Section 4 — neither is optional; Staging cannot function without them.

2 · Environment matrix

Four fully isolated tiers (no shared secrets or data between any of them), distinguished by `APP_ENV`; `NODE_ENV=production` for every tier except Local so that real build optimisations are always exercised before Production.

Tier	`APP_ENV`	Student web	Admin portal	API
Local	`local`	http://localhost:3000	http://localhost:3001	http://localhost:4000
Staging	`staging`	staging.rajyarank.in	admin.staging.rajyarank.in	api.staging.rajyarank.in
Pre-production (UAT)	`preproduction`	preprod.rajyarank.in	admin.preprod.rajyarank.in	api.preprod.rajyarank.in
Production	`production`	rajyarank.in	admin.rajyarank.in	api.rajyarank.in

Domains above are the placeholders already written into the env templates (`.env.staging.example`, `.env.production.example`) — substitute your real domains before use.

Promotion flow

```
feature branch → PR (CI: lint, typecheck, build, unit, integration)
   → merge to main → images built + pushed to GHCR (automatic)
   → rollout to Staging (manual today — see Section 6)
   → manual promote → Pre-production (UAT: acceptance + load + security tests)
   → manual promote (approval) → Production
```

Razorpay stays in **test mode** through Pre-production; **live keys only in Production**.

3 · Infrastructure to provision

Nothing below can be provisioned from inside this engagement — it requires access to your cloud account. This is the concrete shopping list per tier.

Component	Requirement	Notes
PostgreSQL	Managed instance, TLS enforced (`sslmode=require`), automated daily backups + point-in-time recovery	One per tier — never share a database across tiers
Redis	Managed instance (cache / sessions / queues / rate-limit)	`rediss://` (TLS) in Staging/Production per the env templates
Object storage	Private S3-compatible bucket, versioning + lifecycle rules, replicated to a second region	Separate bucket per tier (`rajyarank-staging-private`, `rajyarank-prod-private`)
DNS + TLS	Subdomains per Section 2, valid TLS certs on all of them	Cookies use a shared parent domain per tier (e.g. `.staging.rajyarank.in`) so web + admin share the auth realm
Container host	A place to run 4 containers (`api`, `worker`, `web`, `admin`) pulling from GHCR	**Open decision — see callout below**
Secret store	A secrets manager (cloud-native, Vault, or at minimum a locked-down `.env` on the host)	Never commit real secrets; templates exist for every value needed

Open decision — deploy target. `docs/ci-cd.md` already documents three viable rollout shapes on top of the same GHCR images — a single VM running `docker-compose.deploy.yml`, Kubernetes/ECS running the same images as Deployments/Tasks, or building on the host directly via `docker-compose.prod.yml` (no registry pull needed). The image build/publish pipeline is already wired and works today; only the actual "pull and run on a specific host" step is target-specific and needs your input on where Staging will actually live (a VM you control, or a specific cloud orchestrator).

4 · Action items before the first Staging deploy

These close the two gaps from Section 1. Do them once, in order, before anything else in this plan.

4.1 — Generate the missing Prisma migration history blocking

Against a disposable database that mirrors the current dev schema exactly (not Staging itself):

```
cd apps/api
npx prisma migrate dev --name init
git add prisma/migrations
git commit -m "chore: baseline Prisma migration history"
```

This turns the entire schema (all 8 build phases plus this engagement's additions) into a real, replayable migration the deploy pipeline's `migrate.sh` can actually apply. From this point forward, every schema change must go through `prisma migrate dev` in local development (not `db push`) so the migration history stays authoritative.

4.2 — Bootstrap the first Super Admin blocking

After reference data is seeded (Section 5), the database has zero staff accounts. Recommended: a small one-time script (not part of `seed.ts`, which must stay demo-data-free in production) that creates exactly one `SUPER_ADMIN` staff user from environment variables you control, run once per environment:

```
# one-time, per environment — not part of the regular seed
BOOTSTRAP_ADMIN_EMAIL=you@yourorg.in \
BOOTSTRAP_ADMIN_PASSWORD='' \
node scripts/bootstrap-super-admin.js   # to be written — flag if you want this built now
```

Once this one account exists, every subsequent staff member (Academic Heads, Content Admins, institution staff) is created through the real product flow already built — the Super Admin invites them from the admin portal.

5 · Secrets & configuration checklist

Copy the matching template — `.env.staging.example` or `.env.production.example` — to `.env` on the host / into the CI secret store, and fill every value below from your secret manager. Never commit a filled-in file.

- ☐ `JWT_ACCESS_SECRET` / `JWT_REFRESH_SECRET` — 32+ bytes each, unique per tier (`openssl rand -base64 48`)
- ☐ `FIELD_ENCRYPTION_KEY` — 32 bytes; Production should be KMS-managed, not a flat env value
- ☐ `DATABASE_URL` / `REDIS_URL` — from the instances provisioned in Section 3, TLS-enforced
- ☐ `S3_*` — real bucket + IAM credentials scoped to that bucket only
- ☐ `RAZORPAY_KEY_ID` / `_SECRET` / `_WEBHOOK_SECRET` — **test mode** in Staging/Pre-production, **live mode only in Production**
- ☐ `SMS_PROVIDER=msg91` + `MSG91_TEMPLATE_ID` / `_SENDER_ID` — real DLT-approved template, or OTP stays console-log-only
- ☐ `SMTP_*` — real transactional email provider, or email stays undelivered
- ☐ `GOOGLE_CLIENT_ID` / `_SECRET` — only if Google sign-in should work on that tier
- ☐ `COOKIE_DOMAIN` — the shared parent domain for that tier (e.g. `.staging.rajyarank.in`); `COOKIE_SECURE=true` everywhere except Local
- ☐ `VAPID_*` — only if web push should work on that tier
- ☐ `SENTRY_DSN` — reserved env var; wire it in `apps/api/src/main.ts` (and the Next apps) if you want error tracking before launch

6 · Staging deployment procedure

1. **Provision infrastructure** (Section 3) and fill secrets (Section 5).
2. **Run the two blocking action items** from Section 4 if not already done.
3. **Merge to `main`** — the `deploy.yml` workflow builds and pushes `api`, `worker`, `web`, `admin` images to GHCR, tagged `:sha`, `:main`, `:latest` automatically. Set the repo Actions variable `NEXT_PUBLIC_API_URL` to the Staging API URL first (baked into the web/admin images at build time).
4. **On the target host** (VM, or the equivalent step for your chosen orchestrator per Section 3's callout):

   ```
   export IMAGE_PREFIX=ghcr.io/<owner>/<repo>
   export TAG=latest
   docker compose -f infra/docker/docker-compose.deploy.yml pull
   docker compose -f infra/docker/docker-compose.deploy.yml up -d
   ```

   The `migrate` service runs `prisma migrate deploy` + `constraints.sql` and must finish before `api` / `worker` start (already wired via `service_completed_successfully`).
5. **Seed reference data only** (states, exam bodies, exams, roles, permissions, marketing content, billing plans — no demo users, since `NODE_ENV=production` on this tier):

   ```
   NODE_ENV=production pnpm --filter @rajyarank/api prisma db seed
   ```

6. **Bootstrap the first Super Admin** (Section 4.2) if this is a fresh database.
7. **Point the Razorpay webhook** at `https://api.staging.rajyarank.in/api/v1/webhooks/razorpay` (test mode).
8. **Wire health checks** — `/healthz` (liveness) and `/readyz` (Postgres + Redis reachable) — into whatever is load-balancing/orchestrating the containers.

7 · Post-deploy smoke test

Run through this once per deploy, before telling anyone Staging is ready:

- ☐ `GET /healthz` and `GET /readyz` both 200
- ☐ Log in as the bootstrapped Super Admin; MFA enrolment works
- ☐ Invite one real Academic Head / institution staff member end-to-end (email actually arrives)
- ☐ Create one real Course through the admin portal, add a Subject/Topic/Lesson, publish it
- ☐ As a real student: register (OTP actually arrives via SMS), view the public course page, complete a test purchase with a **Razorpay test card**, confirm entitlement granted
- ☐ Confirm the Razorpay webhook fires and is verified (check API logs for the correlation id)
- ☐ `GET /metrics` reachable only from the internal network / scraper, not the public internet
- ☐ Confirm bilingual EN/HI renders correctly on both web and admin

8 · Promotion to Production — what's different

Item	Staging	Production
Razorpay keys	Test mode (`rzp_test_...`)	**Live mode** (`rzp_live_...`) — only set here
Approval gate	Auto (merge to main)	**Manual approval required** after Pre-production UAT sign-off
Data	Can be reset/reseeded freely	Real user data — irreversible; snapshot before every deploy
Secrets	Tier-specific, lower blast radius	KMS-managed `FIELD_ENCRYPTION_KEY` , tightest IAM scoping
Monitoring	Recommended	**Required** — Sentry wired, alert thresholds set per `docs/monitoring.md`

Otherwise the deploy mechanics are identical to Section 6 — same images, same `migrate.sh` , same compose file, pointed at Production's env file and secrets instead. Do not skip Pre-production (UAT: acceptance + load + security testing) — it exists specifically to catch issues before they reach real users/payments.

9 · Rollback

- **Application:** stateless — redeploy the previous image `TAG` (`docker compose ... pull && up -d` with the old tag, or the equivalent revert on your orchestrator).
- **Database:** migrations are forward-only by design. For a bad migration, restore the pre-deploy snapshot (or use PITR to just before the migration ran) and redeploy the previous image tag together. **Never** hand-edit production data outside a reviewed migration.

10 · Backup, restore & monitoring reference

Backup targets (per `docs/backup-restore.md`)

- PostgreSQL — automated daily backups + PITR; on-demand snapshot immediately before every migration/deploy
- Object storage — bucket versioning + cross-region replication
- Secrets — backed up by the secret manager itself

```
pg_dump "$DATABASE_URL" -Fc -f rajyarank_$(date +%F).dump
pg_restore --clean --if-exists -d "$TARGET_DATABASE_URL" rajyarank_YYYY-MM-DD.dump
```

Restore-test requirement: restore the latest backup into an isolated database, run `prisma migrate status` (expect no drift) plus app smoke tests, verify row counts for users/orders/payments/entitlements/audit, and record RTO/RPO achieved. Target: **RPO ≤ 24h (PITR ≤ 5m), RTO ≤ 1h.** Required before launch, then quarterly.

Monitoring (per `docs/monitoring.md`)

- `GET /healthz` (liveness), `GET /readyz` (Postgres + Redis reachable) — wire to the orchestrator
- `GET /metrics` (Prometheus) — requests by status class, latency, `rr_authz_denied_total` (a security signal) — restrict to the internal network
- Structured JSON logs with a correlation id (`x-request-id`) on every request and 5xx
- Recommended alerts: repeated webhook signature failures, asset-processing failures, spikes in `rr_authz_denied_total`, elevated 5xx rate, queue/job failures

11 · Security & launch checklist (status from the repo's own tracking)

Already implemented (per `SECURITY.md` / `docs/checklists.md`): argon2id password hashing; refresh-token rotation with reuse detection; server-side RBAC + assignment-scope enforcement on every protected endpoint; OTP attempt limits + login rate limiting + lockout; TOTP MFA + AAL2 step-up for high-risk actions; secrets from env only, TOTP secrets encrypted; zod validation on all writes; append-only audit log (DB trigger); signed short-lived upload/playback URLs + per-user watermark + concurrent-session cap; payment/webhook HMAC verification + idempotent webhooks; CSP/HSTS/X-Frame-Options/nosniff headers; health + metrics + correlation-id logging; `pnpm audit` in CI.

Still open before public launch (already tracked, not new findings): malware scanning on upload-complete; nonce-based CSP (drop `'unsafe-inline'` for scripts); container image scanning; external penetration test for IDOR / role-escalation / upload abuse / webhook replay; accessibility + Core Web Vitals review; rate-limit/lockout thresholds reviewed against real traffic; brand/domain/trademark clearance.